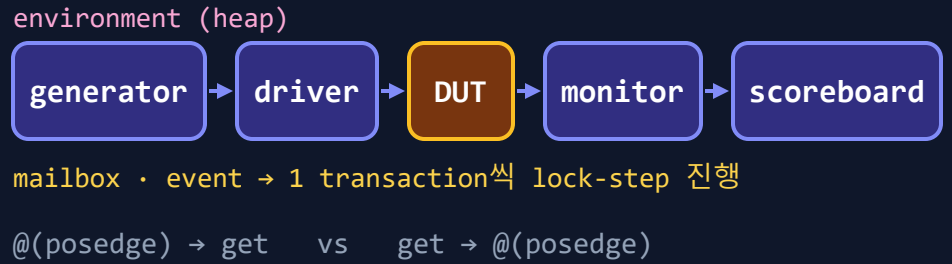
SYSTEMVERILOG · HOMEWORK

Register 8bit TB

driver task 순서 비교 · 이벤트 동기화

tb_register_8bit · class 기반 검증 환경



정안성

2026. 11

구조

gen → drv → mon → scb

mailbox 2개 · event 2개로 transaction 1개씩 순환

초기화

reset 해제 26 ns

preset 15 ns 종료 → driver 첫 drive에서 reset = 0

비교

posedge→get vs get→posedge

driver::run 안의 대기 순서 1곳만 차이

결과

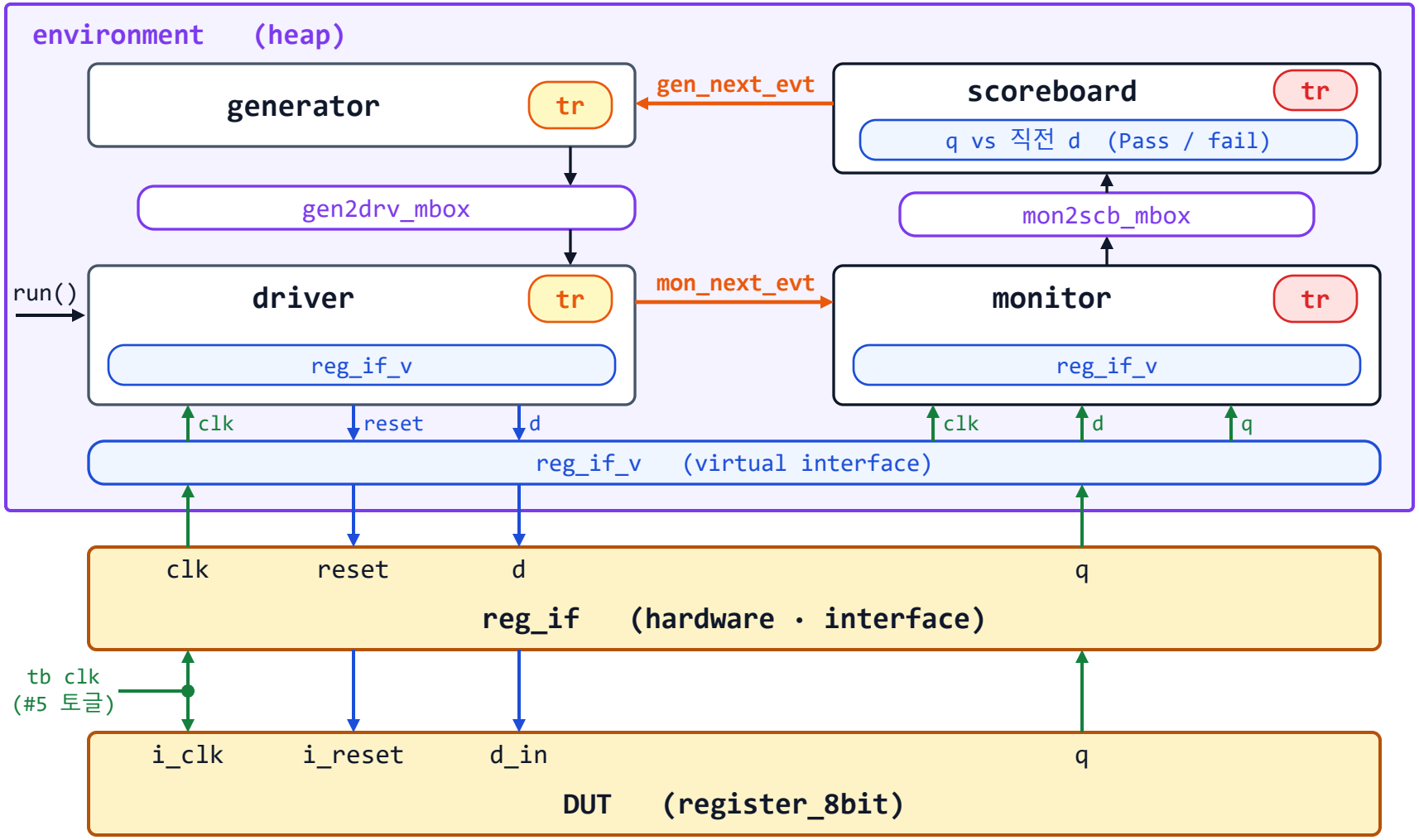
10 ns vs 20 ns / tr

판정 동일 (Pass 255 · skip 1), $finish 2685 ns → 5235 ns

tb_register_8bit 구조: 4개 class를 mailbox 2개 · event 2개로 연결

01

결론 · 4개 class가 mailbox로 데이터를, event로 다음 단계 시작 신호를 전달하는 구조
· driver와 monitor는 하나의 interface를 공유 → driver는 DUT에 값을 넣고, monitor는 DUT 출력을 읽음



generator
· tr = new → randomize → put
· @(gen_next_evt) 대기 → 다음 tr

driver
· @(posedge) → get → #1 → reset · d 구동
· -> mon_next_evt → 다음 edge 대기

monitor
· @(mon_next_evt) → @(posedge)
· d · q 샘플 → mon2scb_mbox.put

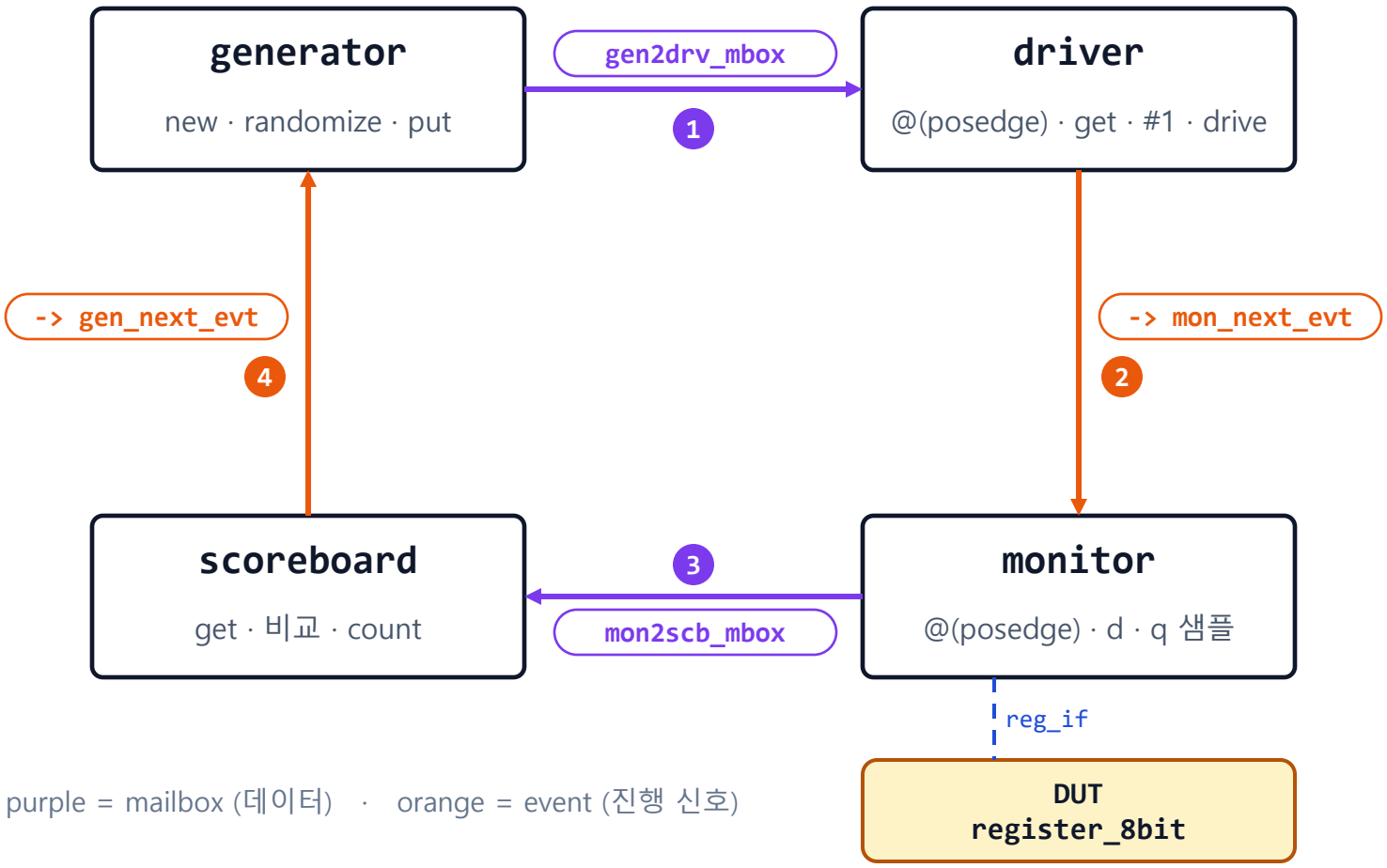
scoreboard
· get → q와 직전 tr.d 비교 (첫 1회 skip)
· Pass / fail count → -> gen_next_evt

environment
· new : mailbox · event · 4 class 생성 · 연결
· run : preset → fork … join_any → #100

event process : 판정이 끝나야 다음 transaction 생성

01

결론 · **transaction은 한 번에 1개씩만 진행 → scoreboard 판정이 끝나야 다음 transaction 생성**
· **generator가 정해진 개수를 모두 만든 뒤 100 ns 대기 → 결과 출력 후 시뮬레이션 종료**



1 **put(tr)**
tr handle → mailbox, GEN은 event 대기

2 **drive + event**
reg_if.d · reset 대입 후 monitor 깨움

3 **sample + put**
다음 posedge에서 d · q 읽기 → mailbox

4 **판정 + event**
Pass / fail count 후 generator 깨움

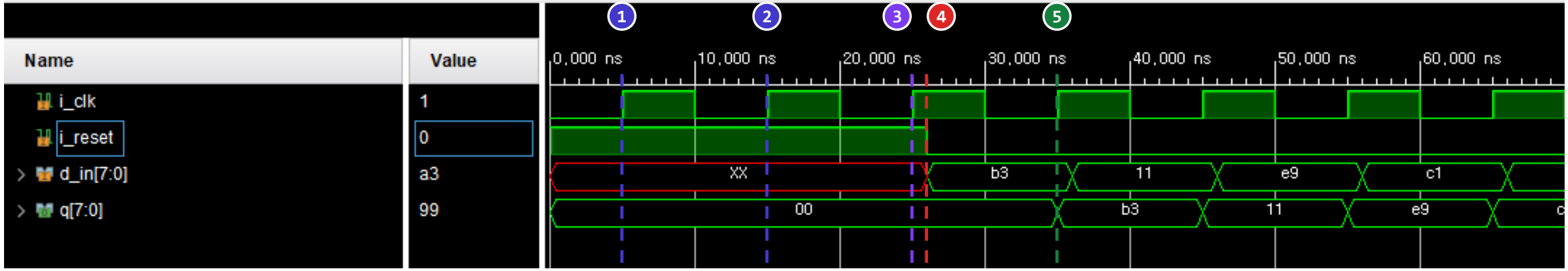
environment::run 시간 흐름 (Case A · run 256)

0 – 15 ns	drv.preset() reset = 1
15 ns	fork gen · drv · mon · scb
2585 ns	gen.run() 종료 → join_any
2685 ns	#100 → count 출력 → $finish

02

reset 해제: preset 종료가 아닌 driver 첫 입력 시점(26 ns)

결론 · reset 해제 시점은 preset 종료(15 ns)가 아닌 driver의 첫 입력 시점(26 ns)
· preset은 reset = 1 상태로 clock 2번만 기다림 → reset = 0은 driver가 첫 데이터를 넣을 때 함께 대입



1 5 ns preset 1번째 edge 2 15 ns preset 종료 3 25 ns driver edge 대기 끝 4 26 ns reset 해제 · 첫 입력 5 35 ns 출력 q 첫 저장

class environment · run()

```
task run(int run_count);  // argument 선언

    drv.preset();
fork
    gen.run(run_count);
    drv.run();
    mon.run();
    scb.run();
join_any
```

class driver · preset()

```
task preset();

    $display("%t : reset = 1 ",$time);
    reg_if_v.reset = 1;
    @(posedge reg_if_v.clk);  1
    @(posedge reg_if_v.clk);  2
    $display("%t : reset_finish ",$time);
endtask
```

class driver · run()

```
task run();
    forever begin
        @(posedge reg_if_v.clk);  3
        gen2drv_mbox.get(tr);
        tr.debug_print("DRV -GET");
        #1;
        tr.debug_print("DRV -PUT");
        reg_if_v.reset = 0;  4
        reg_if_v.d = tr.d;   // DRIVE - D
        -> mon_next_evt;


    end

endtask
```

02

log의 reset = 0: 회로 값이 아닌 transaction 초기값

결론 · **log에 찍힌 reset = 0은 실제 회로 값이 아닌 transaction 객체의 초기값**
· **transaction의 reset은 bit 타입이라 생성 시 0, 이후 값을 넣는 코드 없음 → 실제 회로 reset은 26 ns까지 1**

xsim log

```
==========================================================
 start test
==========================================================
0ns : reset = 1
15ns : reset_finish
15ns :[GEN] d =b3, q = xx , reset = 0
25ns :[DRV -GET] d =b3, q = xx , reset = 0
26ns :[DRV -PUT] d =b3, q = xx , reset = 0
 26ns : -> mon_next_evt !
35ns :[MON - PUT] d =b3, q = 00 , reset = 0
35ns :[SCB] d =b3, q = 00 , reset = 0
35ns -> gen_next_evt
==========================================================
```

① 15ns :[GEN] d =b3, q = xx , reset = 0

class generator · run()

```
task run(int run_count);
    repeat(run_count) begin
        tr = new;
        tr.randomize(); /* W: Ig
        tr.debug_print("GEN");
        gen2drv_mbox.put(tr);
        @(gen_next_evt);
```

② tr = new;

class transaction

```
class transaction;


    bit reset;
    rand bit [7:0] d;
    logic [7:0] q;


    function void debug_print(string name);
        $display("%t :[%s] d =%h, q = %h , reset = %d ",$
time,name,d,q,reset);
    endfunction


endclass
```

③ bit reset;

Variable Init Table

변수	타입	new 직후	값 대입 위치	log 출력
tr.reset	bit (2-state)	0	없음	항상 0 (GEN · DRV · MON · SCB)
tr.d	rand bit [7:0]	0	GEN randomize · MON 샘플	실제 값
tr.q	logic [7:0]	x	MON 샘플만	GEN · DRV = xx, MON · SCB = 실제 값
reg_if.reset	logic	x → 1 (preset)	preset · driver::run	출력 안 함 → 파형으로 확인

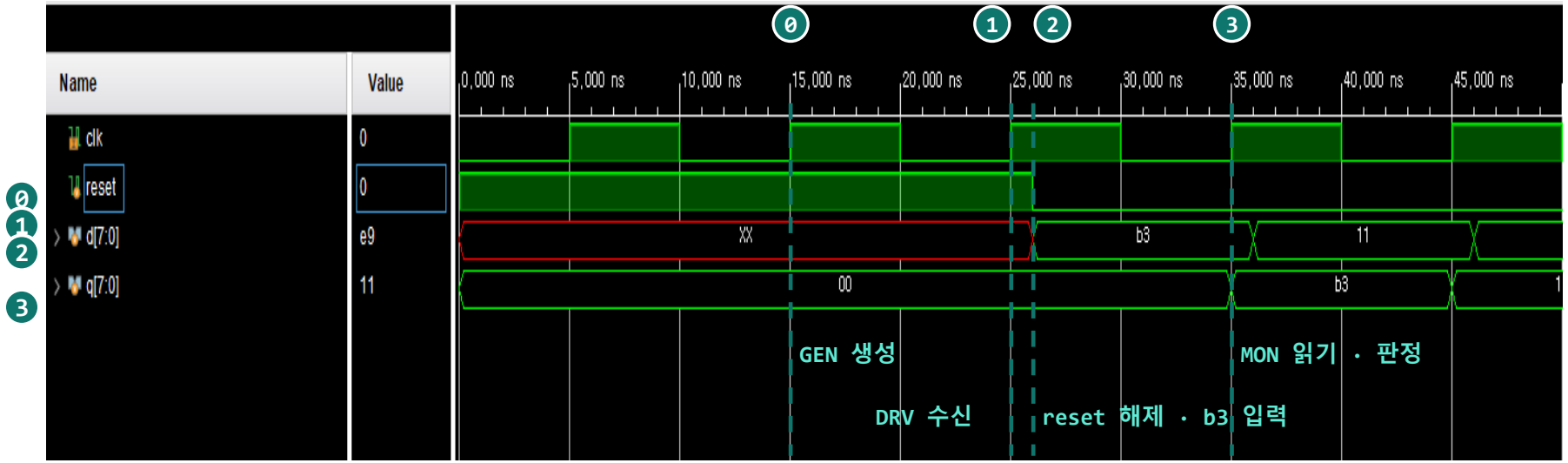
첫 transaction: 15 ns 생성 → 26 ns 입력 → 35 ns 판정

02

결론 · **첫 transaction은 생성부터 판정까지 20 ns (clock 2개) 소요**
· **preset이 끝난 15 ns edge는 이미 지나간 상태 → driver가 다음 edge(25 ns)까지 대기, 이후는 10 ns마다 1개**

xsim log (Case A)

```
==========================================
 start test
==========================================
0ns : reset = 1
15ns : reset_finish
15ns :[GEN] d =b3, q = xx , reset = 0
25ns :[DRV -GET] d =b3, q = xx , reset = 0
26ns :[DRV -PUT] d =b3, q = xx , reset = 0
 26ns : -> mon_next_evt !
35ns :[MON - PUT] d =b3, q = 00 , reset = 0
35ns :[SCB] d =b3, q = 00 , reset = 0
35ns -> gen_next_evt
==========================================
35ns :[GEN] d =11, q = xx , reset = 0
35ns :[DRV -GET] d =11, q = xx , reset = 0
36ns :[DRV -PUT] d =11, q = xx , reset = 0
 36ns : -> mon_next_evt !
45ns :[MON - PUT] d =11, q = b3 , reset = 0
45ns :[SCB] d =11, q = b3 , reset = 0
 45ns : [SCB] Pass  d =  179 , q = 179
45ns -> gen_next_evt
==========================================
```



0 15 ns generator가 데이터(b3) 생성 → mailbox에 전달 후 판정까지 대기

1 25 ns driver가 clock edge 이후 mailbox에서 데이터 수신

2 26 ns 1 ns 뒤 reset 해제 · DUT에 b3 입력 → monitor 깨움

3 35 ns monitor 출력 읽기 → scoreboard 첫 판정 skip → generator 다음 데이터(11) 생성

코드 차이: driver 내부 대기 순서 한 곳

03

결론 · **driver 내부 대기 순서 차이로 인해 발생**
· **Case A는 clock edge를 먼저 기다린 뒤 데이터를 받고, Case B는 데이터를 먼저 받은 뒤 clock edge를 기다림**

Case A · posedge → get

```
task run();
    forever begin
        @(posedge reg_if_v.clk);  1
        gen2drv_mbox.get(tr);     2
        tr.debug_print("DRV -GET"); /* W:
        #1;
        tr.debug_print("DRV -PUT"); /* W:
        reg_if_v.reset = 0;
        reg_if_v.d = tr.d;    // DRIVE - D
        -> mon_next_evt;


    end
```

```
35ns :[GEN] d =11, q = xx , reset = 0
35ns :[DRV -GET] d =11, q = xx , reset = 0
36ns :[DRV -PUT] d =11, q = xx , reset = 0
 36ns : -> mon_next_evt !
45ns :[MON - PUT] d =11, q = b3 , reset = 0
45ns :[SCB] d =11, q = b3 , reset = 0
 45ns : [SCB] Pass  d =  179 , q = 179
45ns -> gen_next_evt
```

@(posedge) → get(tr) → #1 → drive · evt

1. 26 ns drive 후 바로 다음 edge 대기 진입
2. 35 ns edge에서 깨어남 → get 대기 → 같은 35 ns GEN put 수신
3. **#1 → 36 ns drive**

DRV -PUT 26 · 36 · 46 ns

Case B · get → posedge

```
task run();
    forever begin
        gen2drv_mbox.get(tr);     1
        tr.debug_print("DRV -GET"); /* W: Ign
        @(posedge reg_if_v.clk);  2
        //gen2drv_mbox.get(tr);
        //tr.debug_print("DRV -GET");
        #1;
        tr.debug_print("DRV -PUT"); /* W: Ign
        reg_if_v.reset = 0;
        reg_if_v.d = tr.d;    // DRIVE - D
        -> mon_next_evt;


    end
```

```
35ns :[GEN] d =11, q = xx , reset = 0
35ns :[DRV -GET] d =11, q = xx , reset = 0
46ns :[DRV -PUT] d =11, q = xx , reset = 0
 46ns : -> mon_next_evt !
55ns :[MON - PUT] d =11, q = b3 , reset = 0
55ns :[SCB] d =11, q = b3 , reset = 0
 55ns : [SCB] Pass  d =  179 , q = 179
55ns -> gen_next_evt
```

get(tr) → @(posedge) → #1 → drive · evt

1. 26 ns drive 후 mailbox get 대기 진입
2. 35 ns GEN put 수신 → 이 시점부터 edge 대기
3. **35 ns edge 이미 지남 → 45 ns edge + #1 → 46 ns drive**

DRV -PUT 26 · 46 · 66 ns

Case A vs B: 같은 35 ns 수신, 입력 시점은 36 ns vs 46 ns

03

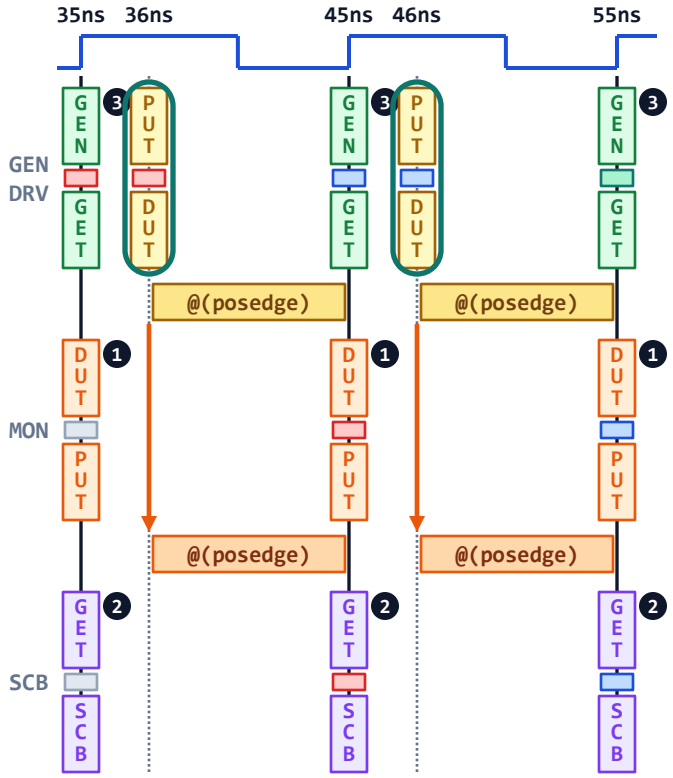
결론 · 같은 35 ns에 데이터를 받아도 Case A는 바로 입력, Case B는 한 clock 뒤에 입력
· 결과적으로 transaction 1개 처리에 Case A는 10 ns, Case B는 20 ns 소요

Case A · @(posedge) → get

```
task run();
    forever begin
        @(posedge reg_if_v.clk); ①
        gen2drv_mbox.get(tr); ②
        tr.debug_print("DRV -GET"); /* W:
        #1;
        tr.debug_print("DRV -PUT"); /* W:
        reg_if_v.reset = 0;
        reg_if_v.d = tr.d;   // DRIVE - D
        -> mon_next_evt;


    end
```

```
35ns :[GEN] d =11, q = xx , reset = 0
35ns :[DRV -GET] d =11, q = xx , reset = 0
36ns :[DRV -PUT] d =11, q = xx , reset = 0
 36ns : -> mon_next_evt !
45ns :[MON - PUT] d =11, q = b3 , reset = 0
45ns :[SCB] d =11, q = b3 , reset = 0
 45ns : [SCB] Pass  d =  179 , q = 179
45ns -> gen_next_evt
```



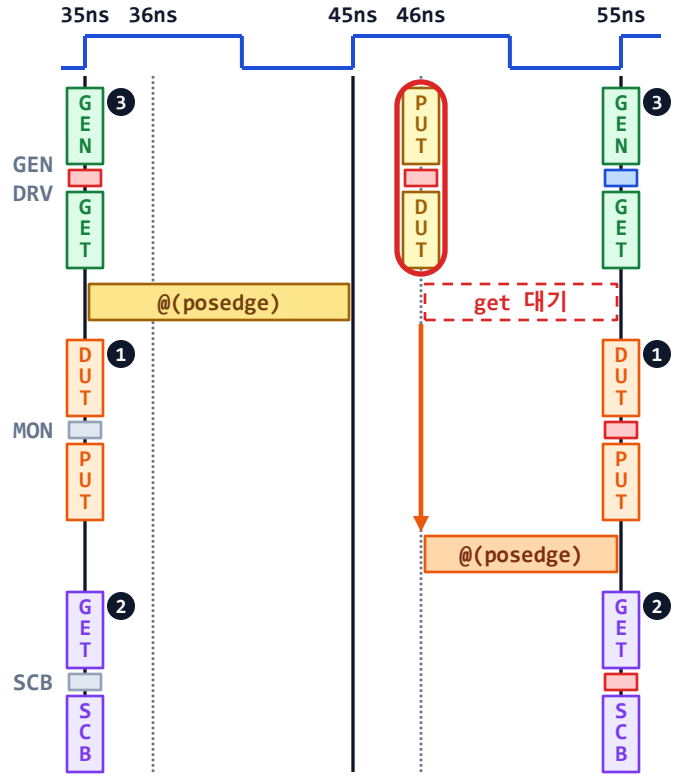
drive 36 · 46 ns → 10 ns / tr

Case B · get → @(posedge)

```
task run();
    forever begin
        gen2drv_mbox.get(tr); ①
        tr.debug_print("DRV -GET"); /* W: Ign
        @(posedge reg_if_v.clk); ②
        //gen2drv_mbox.get(tr);
        //tr.debug_print("DRV -GET");
        #1;
        tr.debug_print("DRV -PUT"); /* W: Ign
        reg_if_v.reset = 0;
        reg_if_v.d = tr.d;   // DRIVE - D
        -> mon_next_evt;


    end
```

```
35ns :[GEN] d =11, q = xx , reset = 0
35ns :[DRV -GET] d =11, q = xx , reset = 0
46ns :[DRV -PUT] d =11, q = xx , reset = 0
 46ns : -> mon_next_evt !
55ns :[MON - PUT] d =11, q = b3 , reset = 0
55ns :[SCB] d =11, q = b3 , reset = 0
 55ns : [SCB] Pass  d =  179 , q = 179
55ns -> gen_next_evt
```



drive 46 ns · 다음 66 ns → 20 ns / tr

①②③ = 같은 timeslot 안 실행 순서 (MON sample → SCB 판정 → GEN put · DRV get)

data : tr b3 tr 11 tr e9 tr c1

원인: clock edge 대기는 대기 시작 이후의 edge만 감지

03

결론

- Case B가 느린 원인은 데이터를 받는 순간 이미 그 clock edge가 지나간 상태이기 때문
- clock edge 대기는 대기를 시작한 이후의 edge만 감지 → 다음 edge까지 1 clock을 더 기다림

35 ns

① clk posedge
35 ns edge

→

② MON
sample · put

→

③ SCB
판정 → gen_next_evt

→

④ GEN
new · put

→

⑤ DRV
get 완료

Case A

@(posedge) 대기 (26 ns ~) | wake | get 대기 (mailbox empty, 0 ns) → get 완료 → 36 ns drive

edge 대기를 먼저 통과 → mailbox 대기만 남음 → 같은 timeslot 안에서 해소

Case B

get 대기 (26 ns ~, mailbox empty) → edge 무관 → get 완료 → 45 ns edge 대기

35 ns edge = 이미 발생 → @(posedge)는 대기 시작 이후의 edge만 검출 → 45 ns edge + #1 → 46 ns drive

Driver State at 35 ns

항목	Case A (@posedge → get)	Case B (get → @posedge)
35 ns edge 순간 DRV 위치	@(posedge) 대기 중 → 깨어남	get 대기 중 → edge 무관
get 완료	35 ns (GEN put 직후, 같은 timeslot)	35 ns (동일)
get 이후 남은 대기	#1	@(posedge) → 45 ns edge, 이후 #1
drive / 1 tr 주기	36 ns / 10 ns	46 ns / 20 ns

결론: clock edge 대기 후 데이터 수신 → 매 clock 1개 처리

04

결론 · **driver는 clock edge를 먼저 기다린 뒤 데이터를 받아야 매 clock 1개씩 처리**
· **판정이 같아도 처리 속도는 다를 수 있음 → event · mailbox로 동기화하는 TB는 같은 시점 안의 실행 순서까지 확인 필요**

결과

1. Case A : 1 tr = 10 ns (1 clock)
2. Case B : 1 tr = 20 ns (2 clock)
3. 판정 동일 : Pass 255 · fail 0 · skip 1

```
total test = 256
pass count = 255
fail count =   0
skip count =   1
```

xsim log · Case A · run(256)

4. $finish 2685 ns vs 5235 ns

원인

1. 35 ns 순서 : edge → MON → SCB → GEN put → DRV get
2. Case A : edge 대기 먼저 통과 → get 즉시 완료
3. Case B : get 완료 후 edge 대기 시작
4. 현재 edge 미검출 → 다음 edge까지 1 clock 대기

고찰

1. **@(event) · @(posedge) = 대기 시작 이후 trigger만 검출**
2. driver = edge 대기 → get 순서로 매 clock 구동
3. event · mailbox 핸드셰이크 TB = 같은 timeslot 안 실행 순서 확인 필요
4. tr 수 증가 시 Case B 손실 = tr당 1 clock 누적

`xsim 실측 : sim_file/tb_register_8bit_posedge_get.sv · tb_register_8bit_get_posedge.sv (run_xsim.sh)`

부록: generator · driver 전체 코드

A

class generator

```
class generator;

    transaction tr;
    mailbox #(transaction) gen2drv_mbox;
    event gen_next_evt;

    function new(mailbox #(transaction) gen2drv_mbox_a, event gen_next_evt);
        this.gen2drv_mbox = gen2drv_mbox_a;
        this.gen_next_evt = gen_next_evt ;
    endfunction



    task run(int run_count);
        repeat(run_count) begin
            tr = new;
            tr.randomize(); /* W: Ignoring return value of non-void function (IEEE 1800-2023 13.4.1)
            tr.debug_print("GEN");
            gen2drv_mbox.put(tr);
            @(gen_next_evt);

            $display("%t -> gen_next_evt ",$time);
            $display("================================================================================");

        end

    endtask

endclass
```

class driver (Case A 순서)

```
class driver;

    transaction tr;
    mailbox #(transaction) gen2drv_mbox;
    event mon_next_evt; /* W: Signal is not used: 'mon_next_evt'

    virtual reg_interface reg_if_v;


    function new(mailbox #(transaction) gen2drv_mbox,  virtual reg_interface reg_if_v,event mon_next_evt)
;
        this.reg_if_v = reg_if_v;
        this.gen2drv_mbox = gen2drv_mbox;
        this.mon_next_evt = mon_next_evt ;
    endfunction

    task run();
        forever begin
            @(posedge reg_if_v.clk);
            gen2drv_mbox.get(tr);
            tr.debug_print("DRV -GET");
            #1;
            tr.debug_print("DRV -PUT");
            reg_if_v.reset = 0;
            reg_if_v.d = tr.d;   // DRIVE - D
            -> mon_next_evt;


        end

    endtask

    task preset();

            $display("%t : reset = 1 ",$time);
            reg_if_v.reset = 1;
            @(posedge reg_if_v.clk);
            @(posedge reg_if_v.clk);
            $display("%t : reset_finish ",$time);
    endtask

endclass
```

부록: monitor · scoreboard 전체 코드

A

class monitor

```
class monitor;

    transaction tr;
    mailbox #(transaction) mon2scb_mbox;
    virtual reg_interface reg_if_v;
    event mon_next_evt;


    function new( mailbox#(transaction) mon2scb_mbox ,
                  virtual reg_interface reg_if_v,
                  event mon_next_evt);

        this.mon2scb_mbox = mon2scb_mbox;
        this.reg_if_v = reg_if_v;
        this.mon_next_evt = mon_next_evt;

    endfunction


    task run();

            forever begin
                @(mon_next_evt);
                $display(" %t : -> mon_next_evt !",$time);
                @(posedge reg_if_v.clk);
                tr = new;
                tr.d = reg_if_v.d;
                tr.q = reg_if_v.q;
                mon2scb_mbox.put(tr);
                tr.debug_print("MON - GET");

            end

    endtask


endclass
```

class scoreboard

```
class scoreboard;

    transaction tr;
    mailbox #(transaction) mon2scb_mbox;
    event gen_next_evt; /* W: Signal is not used: 'gen_next_evt'

    function new(mailbox #(transaction) mon2scb_mbox
                 ,event gen_next_evt );


        this.mon2scb_mbox = mon2scb_mbox;
        this.gen_next_evt = gen_next_evt;
    endfunction

    logic [7:0] expect_data; // 왜 logic으로 선언하는가? DUT 출력이 4상태이기 때문
                             // 출력이 x,z가 나온다면 원하는 출력이 아니다.
                             // 출력이 x

    int d_cnt;
    int pass_cnt;
    int fail_cnt;

    task run();

        forever begin
            // get mail box
            mon2scb_mbox.get(tr);
            // store
            tr.debug_print("SCB"); /* W: Ignoring return value of non-void function (IEEE 1800-2023 13.4

            // pass fail
            if(d_cnt > 0)begin

                    if( expect_data != tr.q)begin
                            $display(" %t : [SCB] fail  d =  %d , q = %d", $time,expect_data,tr.q);
                            fail_cnt ++;
                    end
                    else begin
                            $display(" %t : [SCB] Pass  d =  %d , q = %d", $time,expect_data,tr.q);
                            pass_cnt ++;
                    end
            end

            else begin
                    d_cnt ++ ;

            end

            expect_data = tr.d;


            //evt
            ->gen_next_evt;
        end

    endtask


endclass
```

부록: transaction · environment 전체 코드

A

class transaction

```
class transaction;



    bit reset;
    rand bit [7:0] d;
    logic [7:0] q;


    function void debug_print(string name);
        $display("%t :[%s] d =%h, q = %h , reset = %d ",$
time,name,d,q,reset);
    endfunction



endclass
```

class environment

```
class environment;

    generator gen;
    driver drv;
    monitor mon;
    scoreboard scb;

    mailbox #(transaction) gen2drv_mbox;
    mailbox #(transaction) mon2scb_mbox;
    event gen_next_evt,mon_next_evt; /* W: Signal is not driven: 'gen_next_evt'


    function new(virtual reg_interface reg_if_v);

        gen2drv_mbox = new;
        mon2scb_mbox = new;
        gen = new(gen2drv_mbox,gen_next_evt);
        drv = new(gen2drv_mbox, reg_if_v,mon_next_evt);
        mon = new(mon2scb_mbox,reg_if_v,mon_next_evt);
        scb = new(mon2scb_mbox,gen_next_evt);

    endfunction




    task run(int run_count);  // argument 선언

            drv.preset();
        fork
            gen.run(run_count);
            drv.run();
            mon.run();
            scb.run();
        join_any
        #100;

        $display("total test = %3d",run_count);
        $display("pass count = %3d",scb.pass_cnt);
        $display("fail count = %3d",scb.fail_cnt);
        $display("skip count = %3d",scb.d_cnt);
        $display("================================================================================");

    endtask

endclass
```